Q1. DB-SCAN

Step 1: Visualize Data

In [10]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler

# Generate sample data
centers = [(2, 2), (-2, -2), (2, -2), (-2, 2), (0, 0)]
X, y_true = make_blobs(n_samples=750, centers=centers, cluster_std=0.55, random_state=2)
data = StandardScaler().fit_transform(X)

# Plotting before clustering
plt.figure(figsize=(6, 6))
plt.scatter(X[:, 0], X[:, 1], c=black', grey', marker='x')
plt.title('Sample Data Before Clustering')
plt.grid(True)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

Step 2: Implement K-means Clustering

In [11]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# Generate synthetic data
data, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)

# Calculate WCSS for different numbers of clusters
wcss = []
for i in range(1, 11):  # Testing 1 to 10 clusters
    kmeans = KMeans(n_clusters=i, init='k-means++', max_iter=300, n_init=10,
    random_state=0)
    kmeans.fit(data)
    wcss.append(kmeans.inertia_)  # Inertia is the WCSS for the model

# Plotting the WCSS to observe the 'Elbow'
plt.figure(figsize=(10, 6))
plt.plot(range(1, 11), wcss, marker='o')
plt.title('Elbow Method For Optimal k')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.grid(True)
plt.show()
```

Step 3: Visualize Data After Clustering

In [12]:
```
options, _ = data
kmeans = KMeans(n_clusters=optimal_k, init='k-means++', max_iter=300, n_init=10, random_state=0)
y_kmeans = kmeans.fit_predict(X)

# Plotting the clusters
plt.figure(figsize=(6, 6))
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis')
centers = kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], s=50, c='red', label='Centroids', marker='X'
plt.title('Data After K-means Clustering')
plt.grid(True)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

Step 4: Applied data from step1 with DB-SCAN

In [13]:
```
from sklearn.cluster import DBSCAN

# Applying DBSCAN
dbscan = DBSCAN(eps=0.5, min_samples=5)
y_dbscan = dbscan.fit_predict(X)

# Plotting the DBSCAN results
plt.figure(figsize=(6, 6))
plt.scatter(X[:, 0], X[:, 1], c=y_dbscan, s=50, cmap='viridis')
plt.title('Data After DB-SCAN Clustering')
plt.grid(True)
plt.show()
```

Step 5: Interpret Results

K-mean clustering aim data into k groups. Each data point is assigned to the cluster with the nearest centroid (cluster center).The elbow method is often used to determine the optimal number of clusters by identifying the "elbow point" in the WCSS plot.In our data we use cluster 5. DBSCAN is a density-based clustering algorithm. It groups together data points that are closely packed together (points with many nearby neighbors), marking as outlers points that lie alone in low-density regions (whose nearest neighbors are too far away).In our data we use radius 0.5 and minimum sample 5 .Pro of k means is simple and efficient for large datasets. But it's core is sensitive to the initial placement of centroids. It require specifying the number of clusters before implementing. DBSCAN's pros are robust to outliers, and does not require specifying the numbers of clusters. But it is sensitive to the choice of parameters.It also has the cost of performance degrade with high dimensional data.

Q2. A* algorithm

STEP 1:

In [15]:
```
import heapq

class Node:
    def __init__(self, state, parent=None, g=0, h=0):
        self.state = state
        self.parent = parent
        self.g = g
        self.h = h
        self.f = g + h

    def __lt__(self, other):
        return self.f < other.f

def manhattan_distance(start_state, goal_state):
    """Calculates Manhattan distance heuristic."""
    distance = 0
    for i in range(1, 9):
        for j in range(3):
            for k in range(3):
                if start_state[j][k] == num:
                    x1, y1 = j, k
                    if goal_state[j][k] == num:
                        x2, y2 = j, k
    distance += abs(x1 - x2) + abs(y1 - y2)
    return distance

def get_neighbors(state):
    """Finds possible neighbor states by swapping the 0 tile."""
    neighbors = []
    zero_row, zero_col = next((i, j) for i, row in enumerate(state)
                              for j, val in enumerate(row) if val == 0)

    for dx, dy in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
        new_zero_row, new_zero_col = zero_row + dx, zero_col + dy
        if 0 <= new_zero_row < 3 and 0 <= new_zero_col < 3:
            new_state = [list(row) for row in state]
            new_state[zero_row][zero_col], new_state[new_zero_row][new_zero_col] = new_state[new_zero_row][new_zero_col], new_state[zero_row][zero_col]
            neighbors.append(tuple(tuple(row) for row in new_state))
    return neighbors

def print_board(state):
    for row in state:
        print(' '.join([str(x) if x != 0 else '_' for x in row ]))  # Replace 0 with blank space.
    print()

def astar(start, goal):
    """Performs A* search to find the shortest path."""
    open_set = []
    closed_set = set()

    start_node = Node(start, g=0, h=manhattan_distance(start, goal))
    heapq.heappush(open_set, start_node)

    while open_set:
        current_node = heapq.heappop(open_set)

        if current_node.state == goal:
            path = []
            node = current_node.g
            while node:
                path.append(current_node.state)
                current_node = current_node.parent
                node = node.g
            return path[::-1]  # Return reversed path

        closed_set.add(current_node.state)

        for neighbor_state in get_neighbors(current_node.state):
            if neighbor_state in closed_set:
                continue

            neighbor_node = Node(neighbor_state, current_node, current_node.g + 1, manhattan_distance(neighbor_state, goal))

            if neighbor_node in open_set:
                heapq.heappush(open_set, neighbor_node)

    return None  # No path found

def main():
    """Main function to demonstrate the A* algorithm."""

    start = ((1, 2, 3), (4, 8, 6), (7, 5, 0))
    end = ((1, 2, 3), (4, 5, 6), (7, 8, 0))

    path, total_cost = astar(start, end)

    if path:
        print("Solution found in {len(path) - 1} moves with total cost {total_cost}:\n")
        for i, board in enumerate(path):
            print(f"Move {i}:")
            print_board(board)
    else:
        print("No solution found.")

if __name__ == "__main__":
    main()
```

Solution found in 2 moves with total cost 2:

```
Move 0:
1 2 3
4 8 6
7 5 0

Move 1:
1 2 3
4 5 6
7 8 0
```

STEP 2:

In [16]:
```
import heapq

class Node:
    def __init__(self, state, parent=None, g=0, h=0):
        self.state = state
        self.parent = parent
        self.g = g
        self.h = h
        self.f = g + h

    def __lt__(self, other):
        return self.f < other.f

def manhattan_distance(start_state, goal_state):
    """Calculates Manhattan distance heuristic."""
    distance = 0
    for num in range(1, 9):
        for i in range(3):
            for k in range(3):
                if start_state[i][k] == num:
                    x1, y1 = i, k
                    if goal_state[i][k] == num:
                        x2, y2 = i, k
    distance += abs(x1 - x2) + abs(y1 - y2)
    return distance

def get_neighbors(state):
    """Finds possible neighbor states by swapping the 0 tile."""
    neighbors = []
    zero_row, zero_col = next((i, j) for i, row in enumerate(state)
                              for j, val in enumerate(row) if val == 0)

    for dx, dy in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
        new_zero_row, new_zero_col = zero_row + dx, zero_col + dy
        if 0 <= new_zero_row < 3 and 0 <= new_zero_col < 3:
            new_state = [list(row) for row in state]
            new_state[zero_row][zero_col], new_state[new_zero_row][new_zero_col] = new_state[new_zero_row][new_zero_col], new_state[zero_row][zero_col]
            neighbors.append(tuple(tuple(row) for row in new_state))
    return neighbors

def print_board(state):
    for row in state:
        print(' '.join([str(x) if x != 0 else '_' for x in row ]))  # Replace 0 with blank space.
    print()

def astar(start, goal):
    """Performs A* search to find the shortest path."""
    open_set = []
    closed_set = set()

    start_node = Node(start, g=0, h=manhattan_distance(start, goal))
    heapq.heappush(open_set, start_node)

    while open_set:
        current_node = heapq.heappop(open_set)

        if current_node.state == goal:
            path = []
            node = current_node.g
            while node:
                path.append(current_node.state)
                current_node = current_node.parent
                node = node.g
            return path[::-1]  # Return reversed path

        closed_set.add(current_node.state)

        for neighbor_state in get_neighbors(current_node.state):
            if neighbor_state in closed_set:
                continue

            neighbor_node = Node(neighbor_state, current_node, current_node.g + 1, manhattan_distance(neighbor_state, goal))

            if neighbor_node in open_set:
                heapq.heappush(open_set, neighbor_node)

    return None  # No path found

def main():
    """Main function to demonstrate the A* algorithm with a different starting point."""

    start = ((2, 5, 6), (8, 0, 3), (1, 7, 4))  # New starting configuration
    end = ((1, 2, 3), (4, 5, 6), (7, 8, 0))  # Goal state

    path, total_cost = astar(start, end)

    if path:
        print("Solution found in {len(path) - 1} moves with total cost {total_cost}:\n")
        for i, board in enumerate(path):
            print(f"Move {i}:")
            print_board(board)
    else:
        print("No solution found.")

if __name__ == "__main__":
    main()
```

Solution found in 13 moves with total cost 13:

```
Move 0:
2 5 6
8 _ 3
1 7 4

Move 1:
2 5 6
_ 8 3
1 7 4

Move 2:
2 5 6
1 8 3
_ 7 4

Move 3:
2 5 6
1 _ 3
7 8 4

Move 4:
2 5 6
1 3 _
7 8 4

Move 5:
2 5 6
1 3 4
7 8 _

Move 6:
2 5 6
1 3 4
_ 7 8

Move 7:
2 5 6
_ 3 4
1 7 8

Move 8:
2 5 6
3 _ 4
1 7 8

Move 9:
_ 5 6
2 3 4
1 7 8

Move 10:
5 6
2 3 4
1 7 8

Move 11:
2 5 6
3 _ 4
1 7 8

Move 12:
2 5 6
3 4 _
1 7 8

Move 13:
2 5 6
3 4 7
1 _ 8
```